Softly Constrained MoMos

Carlo Rosso

Contents

1 Introduction 1
2 MoMos and V-Fold MoMos 1
3 Soft MoMos 2
4 Experiments 3
5 Results 3
6 Discussion and Conclusion 9
Bibliography 9

1 Introduction

Many methods in the literature enforce and exploit redundancies in over-parameterized neural networks to compress and simplify them. For example, Mosaic-of-Motifs (MoMos) [1] partitions the weights into small blocks and forces every block to be one of a few reusable motifs. V-Fold MoMos [2] applies MoMos to the mosaic: it partitions the blocks and reduces the number of motifs each block can use. This report adds another piece: Soft MoMos pulls the blocks towards the motifs using the loss, instead of projecting them at the end of each epoch; it increases accuracy and compression rate compared to MoMos and V-Fold MoMos.

2 MoMos and V-Fold MoMos

Let $\boldsymbol{w} \in \mathbb{R}^n$ be the weights and ψ the slicing operator that cuts them into m blocks of shape (r, c), with $s = rc$. A MoMos network is

$$\boldsymbol{w} = \psi^{-1}(\mathcal{Z}[\mathcal{M}]), \quad \mathcal{Z} \in \mathbb{R}^{s \times k}, \quad \mathcal{M} \in [k]^m.$$

The k columns of $\mathcal{Z}$ are the motifs, and the mosaic $\mathcal{M}$ assigns one motif to every block. Storing a block costs $\lceil \log_2 k \rceil$ bits, whatever its size. So larger blocks compress more, and the mosaic dominates the cost.

V-Fold MoMos assign each block position to a fold of $k' \leq k$ motifs. Indices into k' motifs cost fewer bits than indices into k, so the mosaic shrinks.

Both methods project the weights onto the corresponding hypothesis space at the end of every epoch; indeed, they randomly sample the motifs and then they swap each block with its closest motif under the L_2 norm. As shown by [2], the blocks collapse to far fewer distinct motifs than the hard constraint declares at the start of the training. Thus, we can achieve higher compression rates with a postprocessing technique; otherwise, we can preserve higher accuracy, changing the algorithm.

3 Soft MoMos

Soft MoMos applies a soft constraint to the network including the distortion to the loss. At the end of the training, for few epochs the network is recomputed from its representation: mosaic and motifs and the gradient at the level of the blocks is accumulated on the motifs. This is similar to soft weight sharing [3] [4]. Since k is fixed and the weights equal $\mathcal{Z}[\mathcal{M}]$ after the frozen phase, the compression rate is known before training.

The training has three steps (Algorithm 1).

1) **Init**: Sample k blocks of the network at random as the motifs. This is the only randomness, together with the initialization of the network.
2) **Mosaic update**: At the start of every epoch, assign each block to its nearest motif; this gives the mosaic $\mathcal{M}$. For each batch, add the distortion $D = \frac{1}{n}\sum_i \left\| z_{\mathcal{M}_i} - \psi(\boldsymbol{w})_i \right\|^2$ to the loss, as a regularizer. After the step, move every motif to the mean of its blocks. As noted in the thesis [2], the distortion is similar to L_2 regularization, but much weaker in practice.
3) **Frozen phase**: In the last epochs (e.g. the last 25%), fix $\mathcal{M}$ and compute the network as $\boldsymbol{w} = \psi^{-1}(\mathcal{Z}[\mathcal{M}])$. The gradient of a motif is the sum of the gradients of its blocks, and only $\mathcal{Z}$ is trained.

Both motifs and mosaic are learnt throughout the training, and so there is a small overhead during the training, since the distortion is computed for every batch.

Soft MoMos Training

Input: weights $\boldsymbol{w}$, block shape (r, c), motifs k, epochs T, frozen fraction p (`frozen_pct`), distortion scale λ
$\mathcal{Z} \leftarrow \{0\} \cup \{ \text{ sample } \ k-1 \ \text{ blocks from } \ \psi(\boldsymbol{w})\}$
for epoch in 1, …, $(1-p) \cdot T$ **do** ▷ soft phase
  $\mathcal{M}_i \leftarrow \arg\min_{j \in [k]} \left\| z_j - \psi(\boldsymbol{w})_i \right\|$ for every block i
  for batch in 1, …, batches **do**
    update $\boldsymbol{w}$ with a gradient step on $\mathcal{L}_{\text{batch}} + \lambda \cdot D$
    $z_j \leftarrow \text{mean } \{\psi(\boldsymbol{w})_i : \mathcal{M}_i = j\}$ for every $j \in [k] \setminus \{0\}$ ▷ motifs follow the blocks
for epoch in 1, …, $p \cdot T$ **do** ▷ frozen phase: $\mathcal{M}$ is fixed
  for batch in 1, …, batches **do**
    $\boldsymbol{w} \leftarrow \psi^{-1}(\mathcal{Z}[\mathcal{M}])$
    $\nabla z_j \leftarrow \sum_{i:\mathcal{M}_i = j} \nabla_{\psi(\boldsymbol{w})_i} \mathcal{L}_{\text{batch}}$ for every $j \in [k] \setminus \{0\}$
    update $\mathcal{Z}$ with a gradient step ▷ only $\mathcal{Z}$ is trained
return $\mathcal{Z}$, $\mathcal{M}$

4 Experiments

We trained Soft MoMos for 100 epochs to find the best

- frozen_pct per block size;
- scale of the distortion regularizer.

We also want the best block size for a fixed $k = 256$. We chose $k = 256 = 2^8$ because it is the most convenient value in practice: it corresponds to the type `uint8`. For the MLP, this corresponds to the capacity $(k \cdot s)/n = 0.03\% \cdot s$. We fix k because the original paper [1] points out that, at a fixed capacity, a smaller block size is better. Thus, a fixed k means the capacity increases with the block size, and the compression rate.

We searched block shapes $(r, c) \in \{1, 2\} \times \{2, 3, 4, 5\}$. This report compares four configurations: QAT, Soft MoMos, MoMos, and V-Fold MoMos (written Fold in the figures). Shapes are written (r, c).

After the initial search, we repeated the training of the best configurations for 200 epochs with 5-fold cross-validation. Finally we report the compression ratio as $r_\varphi = (n \cdot q)/(ksq + \lceil n/s \rceil \cdot \lceil \log_2 k \rceil)$, where q is the bit width of a weight.

5 Results

- Soft MoMos seems to beat the previous Pareto front on the MLP (Figure 1, Table 1). The breakdown of the compressed size into motifs and mosaic is in Figure 2.
- At the end of the training (at frozen_pct), the training accuracy drops while the validation accuracy rises (Figure 3).
- The validation accuracy stays low during the soft phase and jumps at the freeze (Figure 6).

- Soft MoMos induces a good amount of sparsity (Figure 4). This is reasonable, since it acts like an L_2 penalty on weights close to 0.
- BDM [5] and QuBD [6] show that complexity increases after projection onto the MoMos space (Figure 7, Figure 8). This seems to contradict the reduced overfitting.
- The entropy smoothly decreases after the projection (Figure 9), thus the increase in BDM and QuBD may not come from the entropy factor. We cannot explain this yet, and bugs are always possible.
- The distortion of Soft MoMos is orders of magnitude bigger than that of MoMos and V-Fold MoMos, and it has not converged by the end of the soft phase (Figure 5).

Method	**Shape**	r_φ	**Test acc. (%)**	**Pareto**
QAT	8 bit	4.0	58.0 ± 0.5	✓
	4 bit	8.0	57.5 ± 0.4	✓
MoMos	(1,2)	4.7	57.5 ± 0.4	
	(1,2)	9.1	54.4 ± 0.5	
	(2,2)	9.5	54.4 ± 0.2	
	(2,2)	18.1	50.4 ± 0.4	
	(2,4)	30.6	47.9 ± 0.5	
	(2,4)	30.6	47.6 ± 0.4	
V-Fold MoMos	(2,2)	11.8	53.2 ± 0.4	
	(2,2)	17.3	51.4 ± 0.4	
	(1,4)	26.3	49.3 ± 0.1	✓
	(1,4)	39.3	41.1 ± 1.2	✓
Soft MoMos	(1,3)	11.9	55.8 ± 0.5	✓
	(2,2)	15.7	54.6 ± 0.4	✓
	(1,5)	19.5	52.8 ± 0.5	✓
	(2,3)	23.1	52.5 ± 0.4	✓
	(2,5)	36.2	48.6 ± 0.7	✓

Table 1: Values referred to Figure 1, test accuracy (mean $\pm$ std over 5 folds) and compression rate. V-Fold MoMos lists its primary block shape.

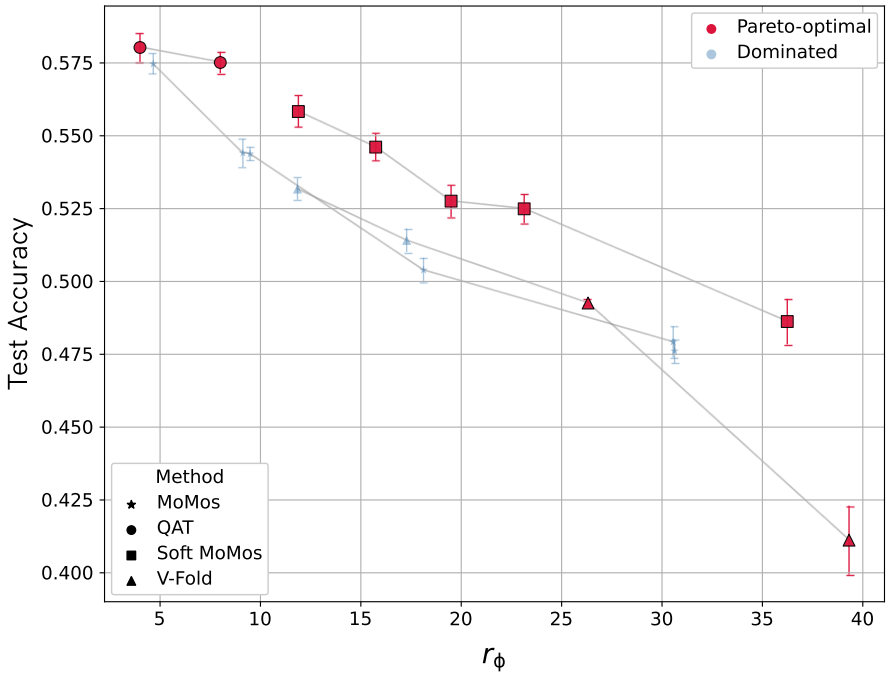


Figure 1: CV-verified test accuracy against compression rate r_φ, for all methods.

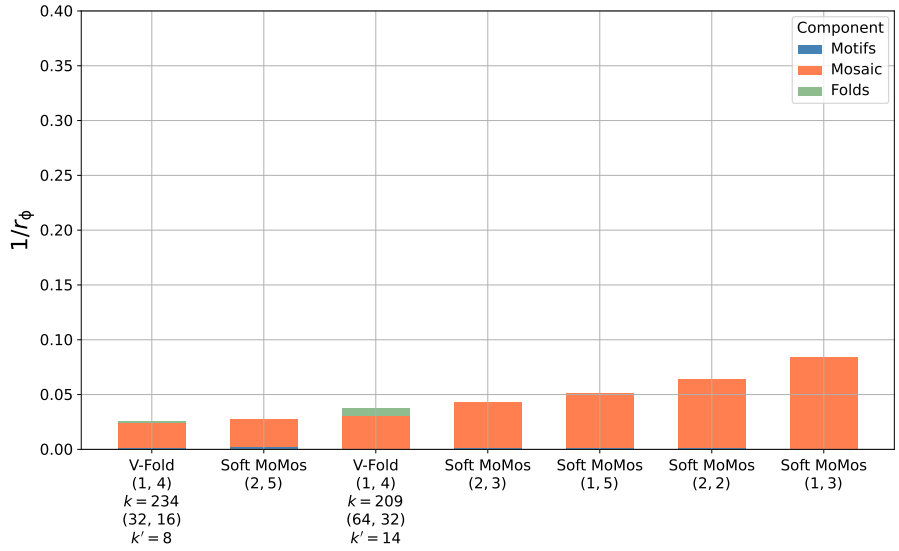


Figure 2: Compressed size of the Pareto-optimal configurations, split into motifs and mosaic.

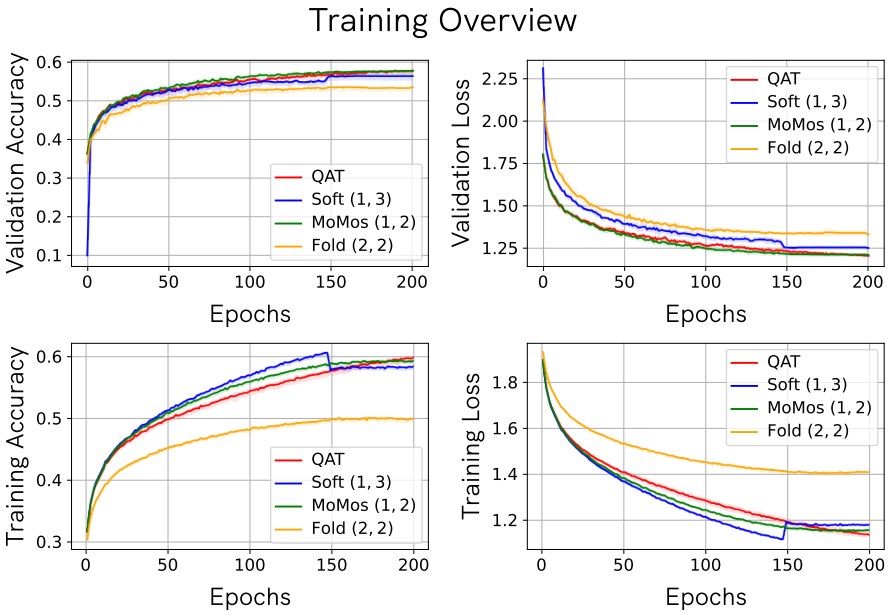


Figure 3: Training overview of QAT, Soft (1,3), MoMos (1,2) and Fold (2,2); the shapes, hence the compression rates, differ.

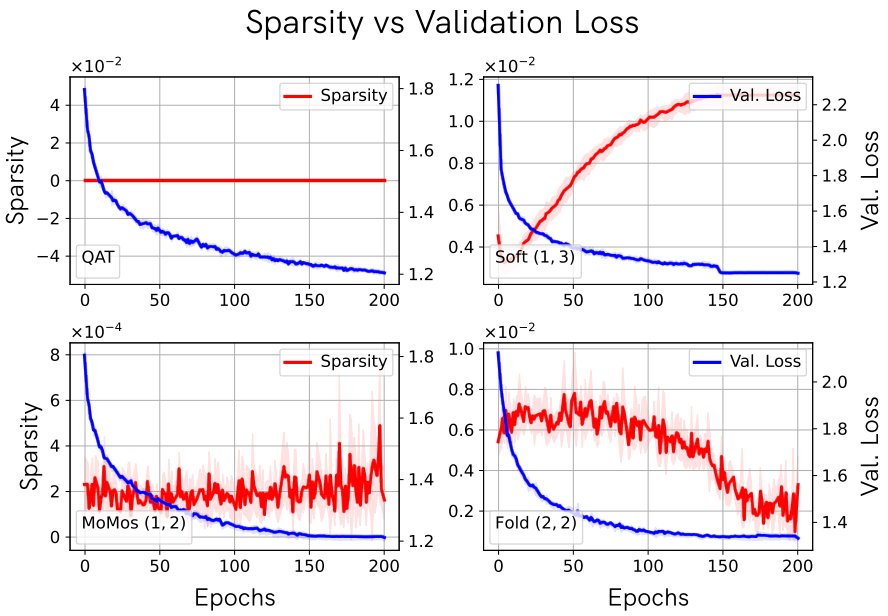


Figure 4: Sparsity against validation loss over training.

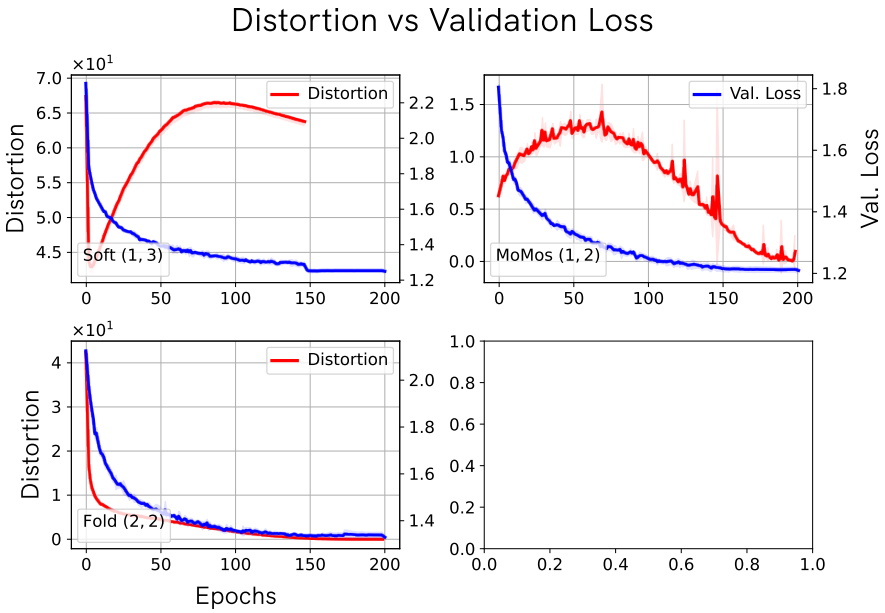


Figure 5: Distortion against validation loss over training.

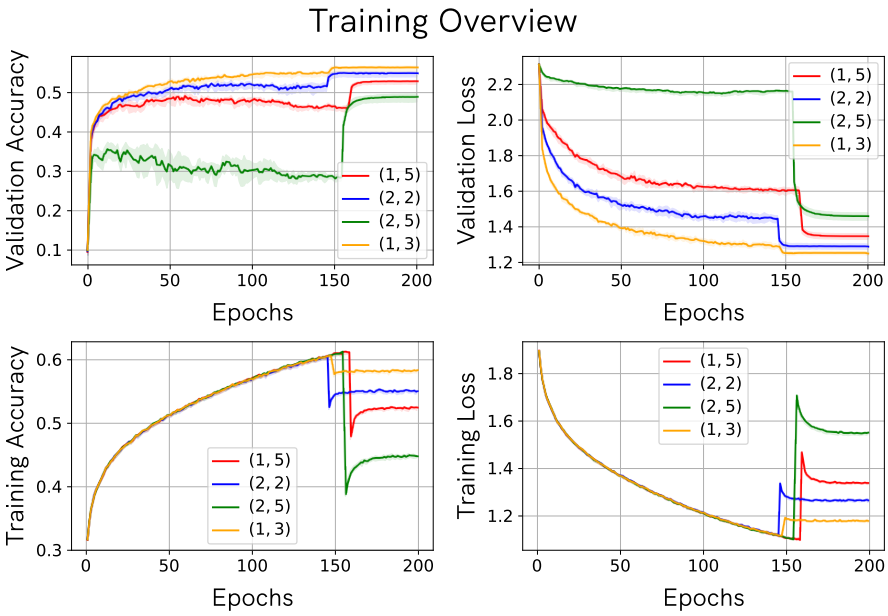


Figure 6: Training overview of 4 Soft MoMos experiments.

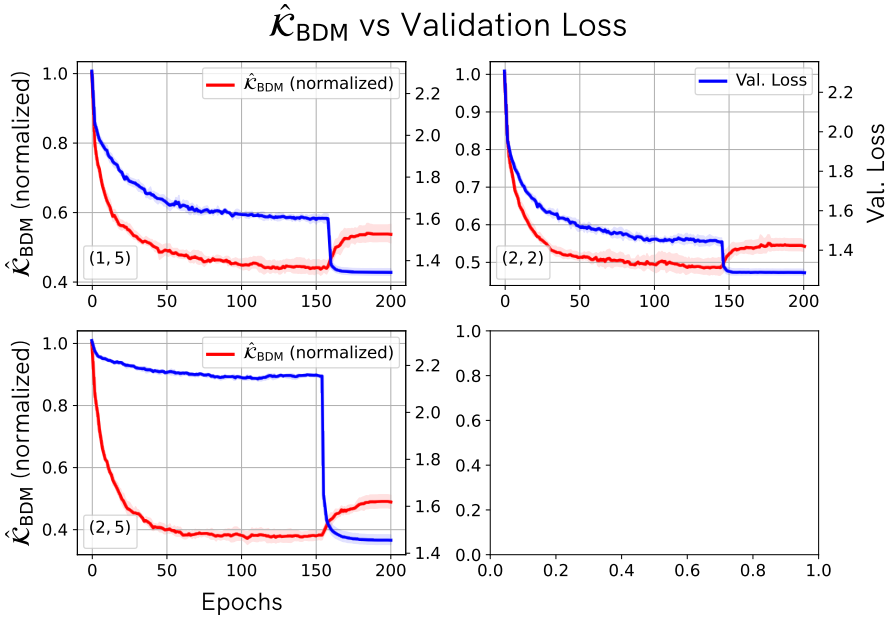


Figure 7: BDM complexity against validation loss, for four Soft MoMos shapes.

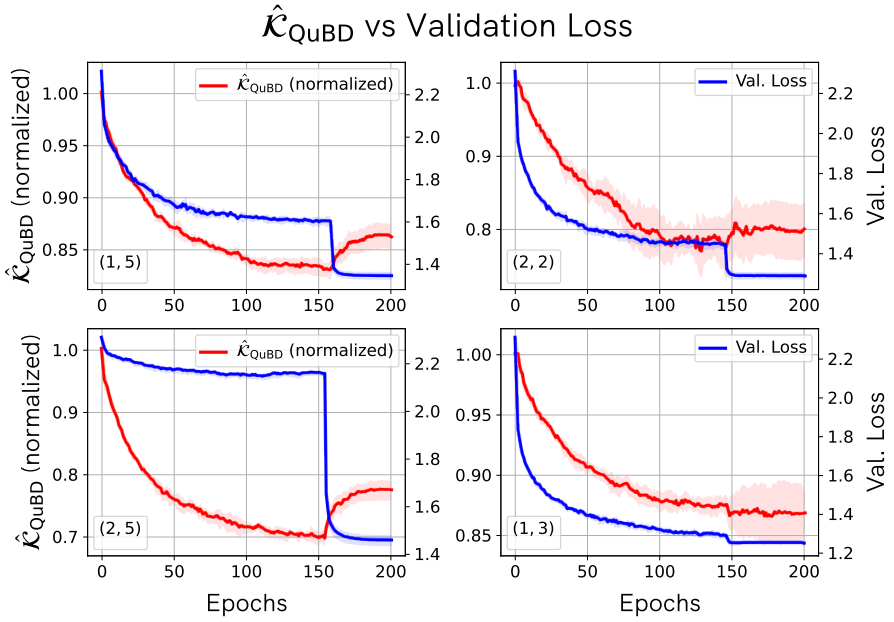


Figure 8: QuBD complexity against validation loss, for four Soft MoMos shapes.

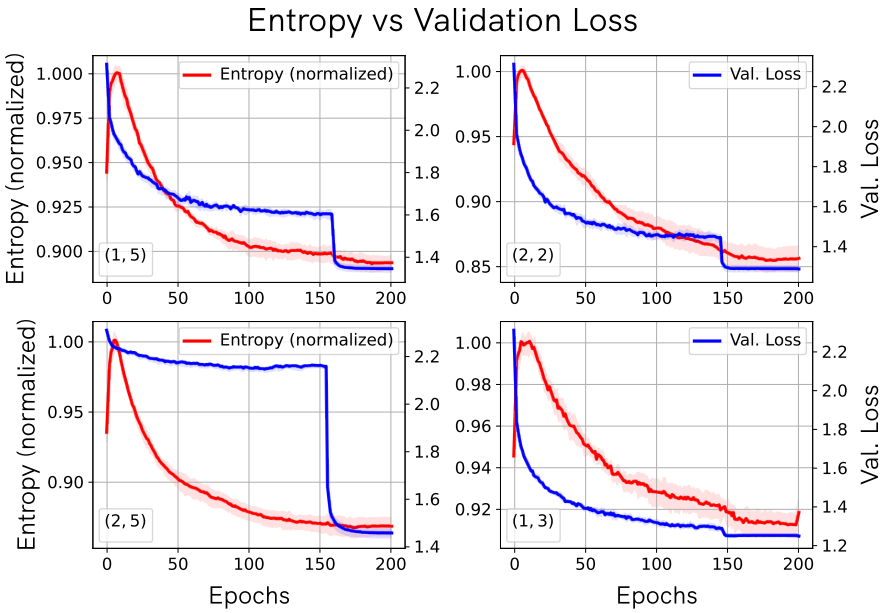


Figure 9: Entropy against validation loss, for four Soft MoMos shapes.

6 Discussion and Conclusion

We find this result interesting because it invites deeper thought about the compression constraint enforced by MoMos. We do not claim that the proposed algorithm is final, but it shows that the hard MoMos constraint allows a rich hypothesis space. The MoMos algorithm in [1] has a very strong constraint that is not fully exploited: we could either compress more or preserve higher accuracy.

The regularization could be strengthened, perhaps with an L_1 norm. This might reduce the distortion and stabilize training, avoiding the jump in training and validation accuracy that the MoMos projection causes at the end.

It is also important to highlight that the best hyper-parameters were found using optuna on a training of 100 epochs, and then the training was extended to 200 epochs. It means that the `frozen_pct` is biased on the searching space. Therefore, an algorithm that is robust to the number of epochs could enhance the performance.

We also highlight that the MLP model on CIFAR10 tends to overperform the quality of the method.

Bibliography

[1] P. Bakhtiarifard, T. Chen, J. Wenshøj, E. B. Dam, and R. Selvan, “Algorithmic Simplification of Neural Networks with Mosaic-of-Motifs.” [Online]. Available: http://arxiv.org/abs/2602.14896

[2] C. Rosso, P. Bakhtiarifard, and R. Selvan, “Neural network simplification through Kolmogorov Complexity bounding,” Master's thesis, 2026. [Online]. Available: https://github.com/rossoc/V-Fold-MoMos/blob/main/master_thesis.pdf

[3] S. J. Nowlan and G. E. Hinton, “Simplifying Neural Networks by Soft Weight-Sharing,” *Neural Computation*, vol. 4, no. 4, pp. 473–493, 1992, doi: 10.1162/neco.1992.4.4.473.

[4] K. Ullrich, E. Meeds, and M. Welling, “Soft Weight-Sharing for Neural Network Compression,” in *International Conference on Learning Representations*, 2017.

[5] H. Zenil, F. S. Toscano, and N. Gauvrit, “The Block Decomposition Method,” *Methods and Applications of Algorithmic Complexity: Beyond Statistical Lossless Compression*. Springer, Berlin, Heidelberg, pp. 125–149, 2022. doi: 10.1007/978-3-662-64985-5_6.

[6] P. Bakhtiarifard, S. N. Wilson, M. Afifi, J. Wenshøj, and R. Selvan, “Characterizing Learning in Deep Neural Networks using Tractable Algorithmic Complexity Analysis.” [Online]. Available: http://arxiv.org/abs/2605.15551